ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ

ΣΧΟΛΗ ΗΜΜΥ

Τηλεπικοινωνιακά Συστήματα Ι

Παράδωση 1ης εργασίας

Ημερομηνία Παράδοσης: 16/3/2026

Ομάδα 101

Κύρκος Κωνσταντίνος 2022030112

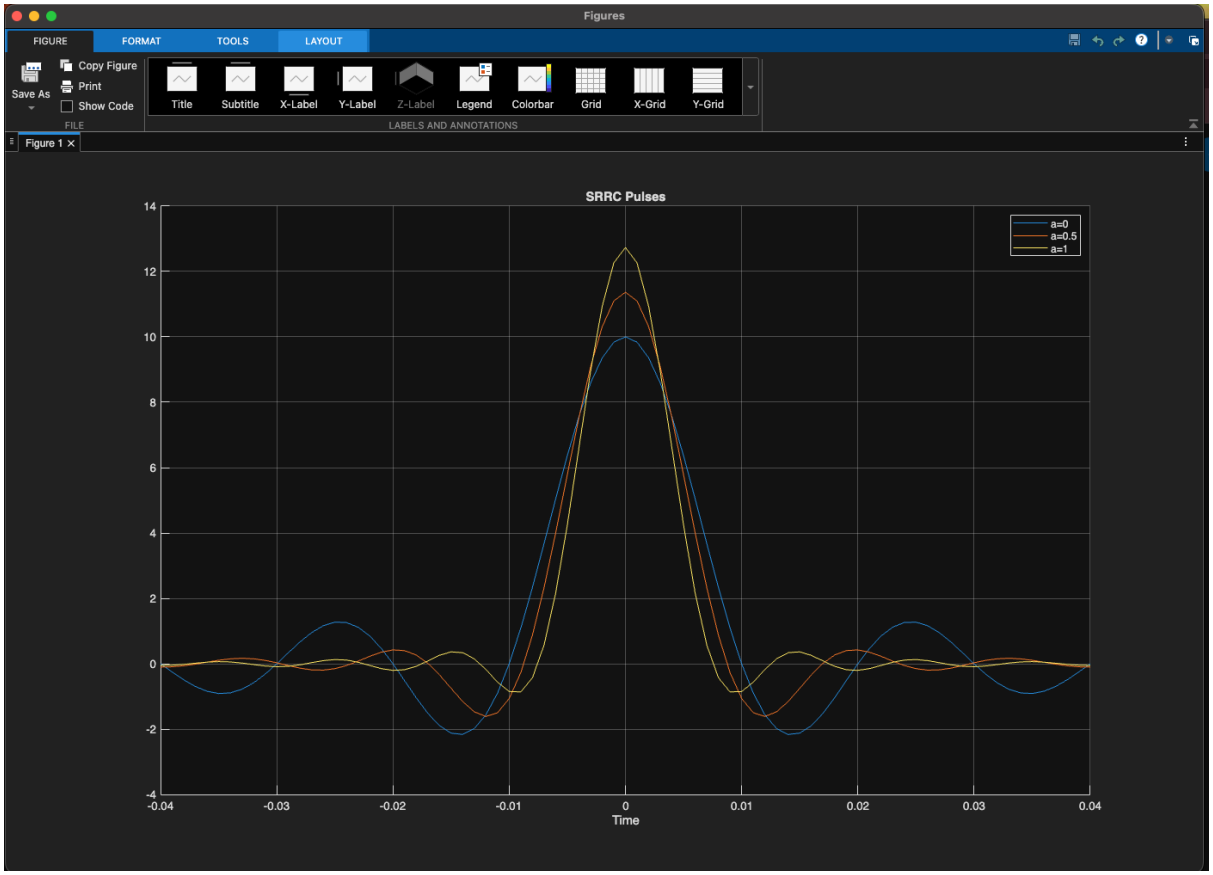
A.1

Construct SRRC pulses φ(t).

(5) Plot the pulses on the same figure (a common plot) with an appropriate time axis, for T = 10−2sec, over = 10, A = 4, and a = 0,0.5,1.

(5) What do you observe about the rate at which the pulse amplitude "decays" as the absolute time |t| increases, with respect to the value of a? Which pulse decays faster?

Solution:



Based on the generated plots, we observe that the rate at which the pulse amplitude φ(t) decays is directly proportional to the roll-off factor α. Specifically, as the value of α increases, the pulse exhibits a faster decay rate, leading to smaller sidelobes and a more compact pulse shape. Consequently, the pulse with α=1 decays the fastest, whereas the pulse with α=0 (sinc pulse) demonstrates the slowest decay.

The MATLAB code is shown below:

/Users/kyrkos/Documents/MATLAB/MATLAB_LAB1/AskisiA.m

```
clear all;
close all;

T = 0.01;                   %Period
over = 10;                  %Over factor
Ts = T/over;                %Sampling Period
A = 4 ;
a = [0, 0.5, 1];            %Roll-off Factor
phi = {};                   %Initialization of truncated SRRC pulse
t = 0;                      %Initialization of time

%Question A1

figure(1)
grid on;
hold on;

%Set variables for each 'a' and plot
for i=1:length(a)
    [phi{i}, t] = srrc_pulse(T, over, A, a(i));
    plot(t,phi{i});
end
legend("a=0", "a=0.5","a=1")
xlim([-A*T A*T])
title("SRRC Pulses")
xlabel("Time")
hold off;
```

A.2

Using the functions fft and fftshift, compute the Fourier transforms (F) of the pulses from the previous step at Nf equally spaced frequency points over [−Fs/2 , Fs/2) (e.g., Nf = 1024, 2048).

Plot the energy spectral density |Φ(F)|^2 of these pulses:

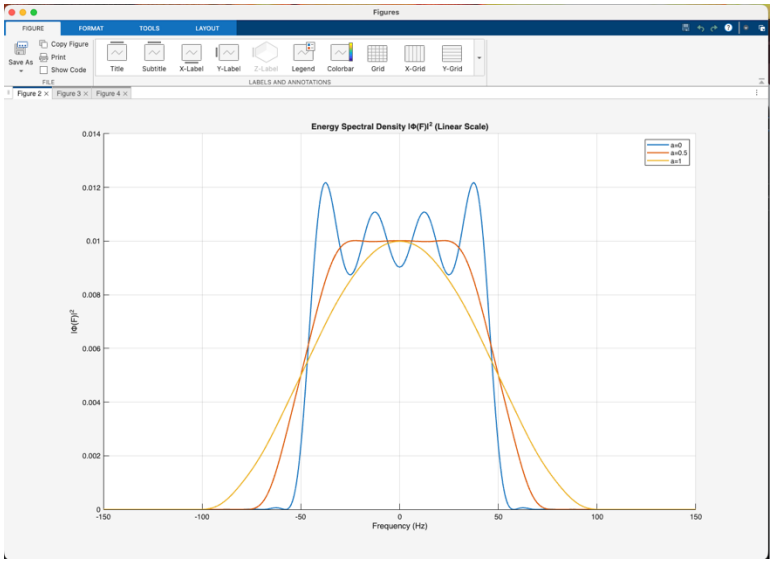
(a) (5) on a common plot,

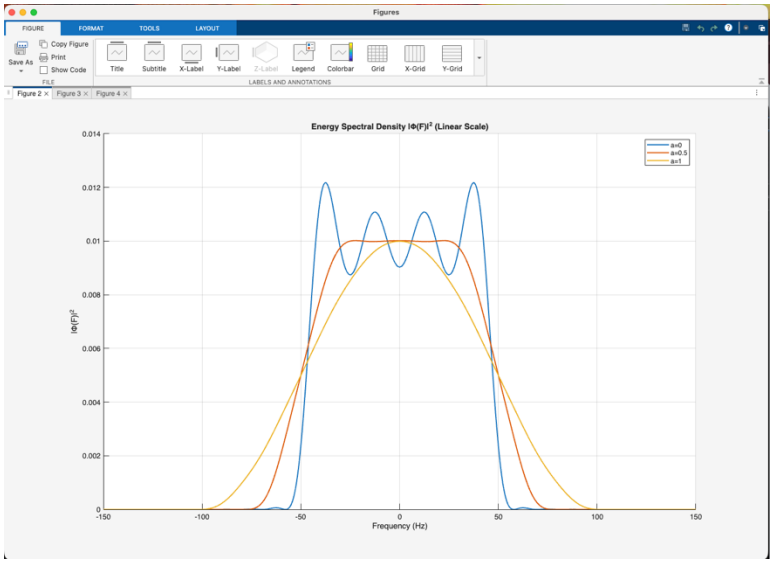
(b) (5) on a common semilogy (note that semilogy lets you study the values of |Φ(F)|^2 over ranges where they are very small, which cannot be done easily with a linear plot).

Solution:

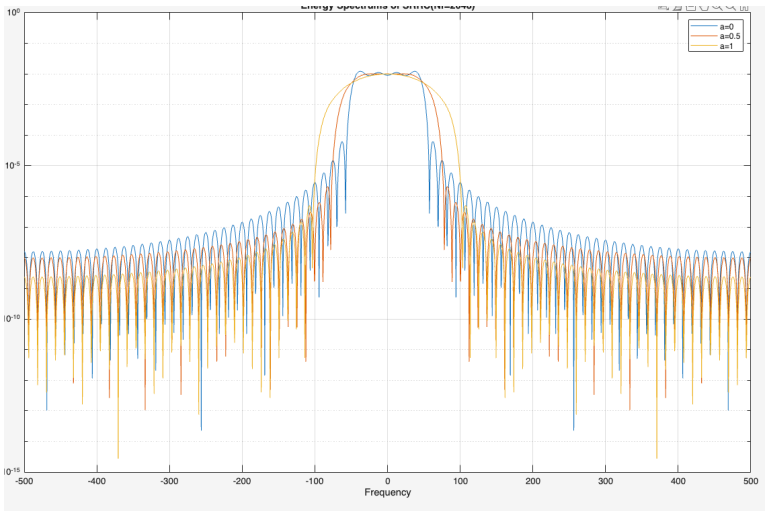
(Left is 1024 Right is 2048)

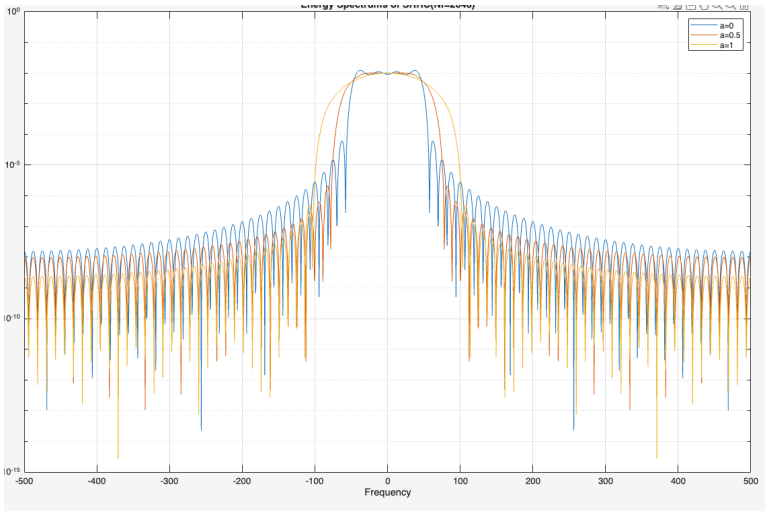
a)





b)





The MATLAB code is shown below:

```
%Question A2

Fs = 1/Ts;  %Sampling Freq
%Nf = 1024;
Nf = 2048;
phi_F = {};
phi_energy = {};

f_axis =  linspace(-Fs/2,(Fs/2-Fs/Nf),Nf);  %Freq range

figure(2)

hold on; grid on;
for i=1:length(a)
    phi_F{i} = fftshift(fft(phi{i}, Nf) * Ts);
    phi_energy{i} = abs(phi_F{i}).^2;
    plot(f_axis, phi_energy{i}, 'LineWidth', 1.5);
end
legend("a=0", "a=0.5", "a=1");
title('Energy Spectral Density |\Phi(F)|^2 (Linear Scale)');
xlabel('Frequency (Hz)');
ylabel('|\Phi(F)|^2');
xlim([-1.5/T, 1.5/T]);
hold off;

%Set energy spectrum using semilogy

figure(3)
for i=1:length(a)
    semilogy(f_axis, phi_energy{i});
    hold on;
end
grid on;
legend("a=0", "a=0.5","a=1");
xlim([-Fs/2 Fs/2])
title('Energy Spectrums of SRRC(Nf=2048)');
xlabel('Frequency');
ylabel('Logarithmic');
hold off;
```

A.3

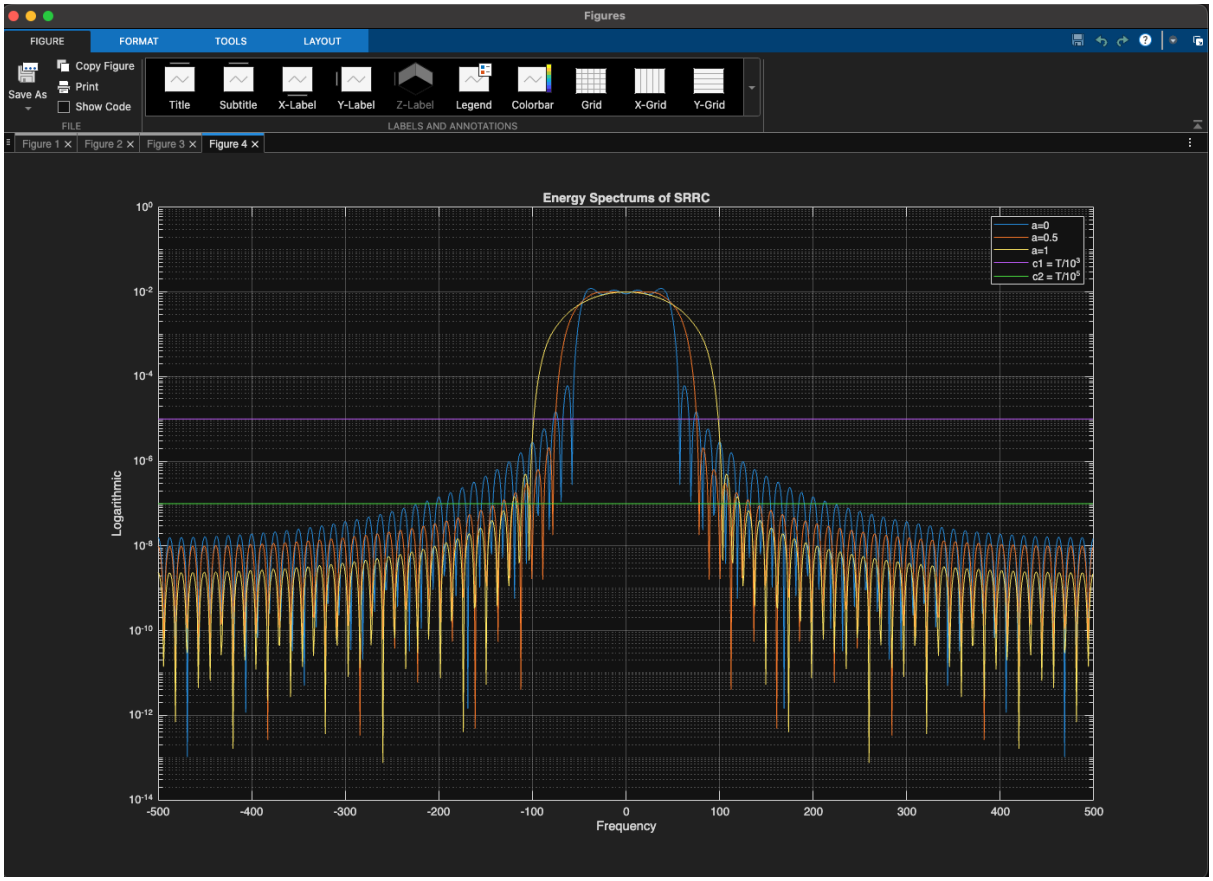
The theoretical bandwidth of infinite-duration pulses is BW = (1+a)/2T .

(5) Compute the theoretical bandwidth for each of the three pulses.

a	0	0,5	1
BW	50	75	100

(5) In practice, since truncated pulses have (theoretically) infinite bandwidth, a more practical definition of bandwidth is needed. On the common semilogy plot from question A.2, draw a horizontal line with value c (e.g., c = T/10^3) and consider values below this line to be "practically zero." Under this definition, what is the approximate bandwidth of the three pulses (the "Zoom in" feature will be helpful). Which pulse is more bandwidth-efficient? How does the bandwidth change if c = T/10^5 ? In this case, which pulse is more efficient?

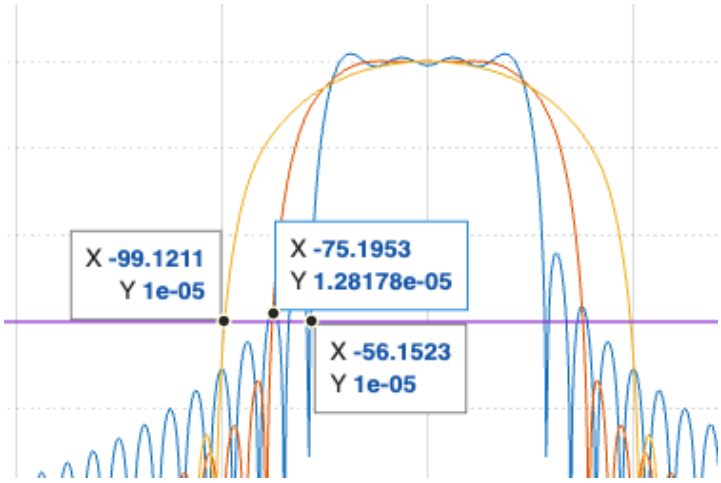
Using the MATLAB Plot shown below the real world BW results were approximated:



For: T/10^3:

a	0	0,5	1
BW	56.1523	75.1953	99.1211

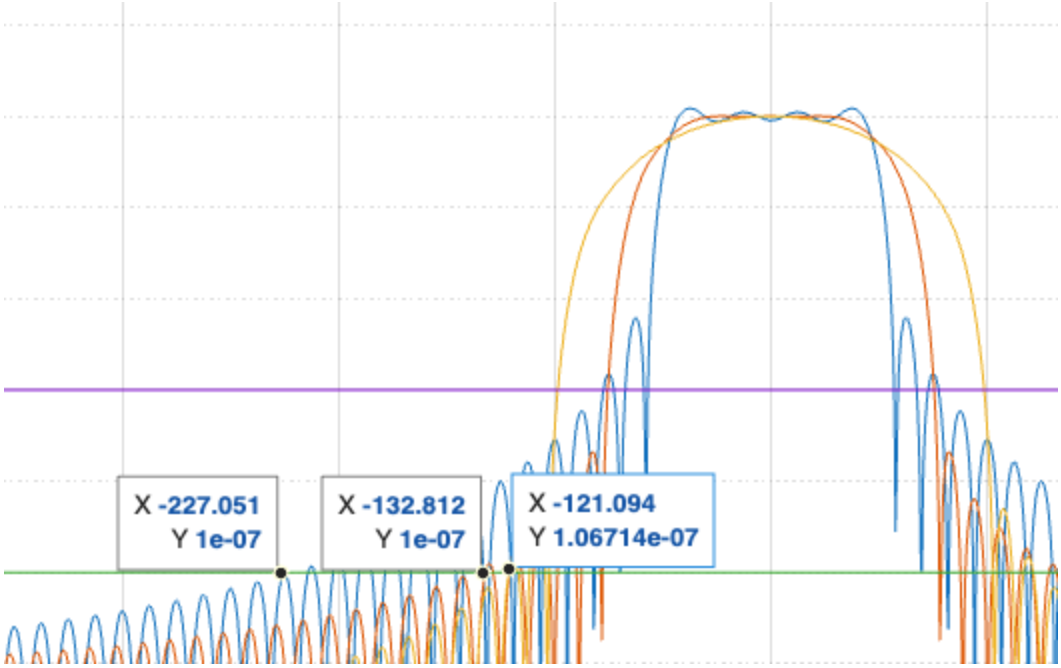
The results are also shown below:



For: T/10^5:

a	0	0,5	1
BW	227,051	132,812	121,094

The results are also shown below:



Conclusion:

Practically, the bandwidth is determined by the frequency at which the Energy Spectral Density drops below a threshold . When using a higher threshold (, c = T/10^3), the pulse with a = 0 appears more bandwidth-efficient due to its narrower main lobe. However, as the threshold becomes stricter (, c = T/10^5), the pulse with a = 1 often becomes more efficient. despite its wider main lobe, its sidelobes decay much faster, crossing the lower threshold at a closer frequency than the α=0 pulse. This demonstrates that for truncated pulses, the optimal roll-off factor is not fixed but depends on the specific out-of-band interference requirements of the system.

The MATLAB code is shown Below:

```
%Question 3

c1 = zeros(1,length(f_axis))+T/10^3;
c2 = zeros(1,length(f_axis))+T/10^5;

figure(4);

for i=1:length(a)
    semilogy(f_axis, phi_energy{i});
    hold on;
end
grid on;
title('Energy Spectrums of SRRC');
xlabel('Frequency');
ylabel('Logarithmic');
plot(f_axis, c1);
plot(f_axis, c2);
legend("a=0", "a=0.5","a=1", "c1 = T/10^3","c2 = T/10^5");
xlim([-Fs/2 Fs/2])
hold off;
```

B.1

For T = 10^−2, A = 4, a = 0, 0.5, 1, and k = 0, 1, . . . , 2A (you will obtain corresponding results for negative k)

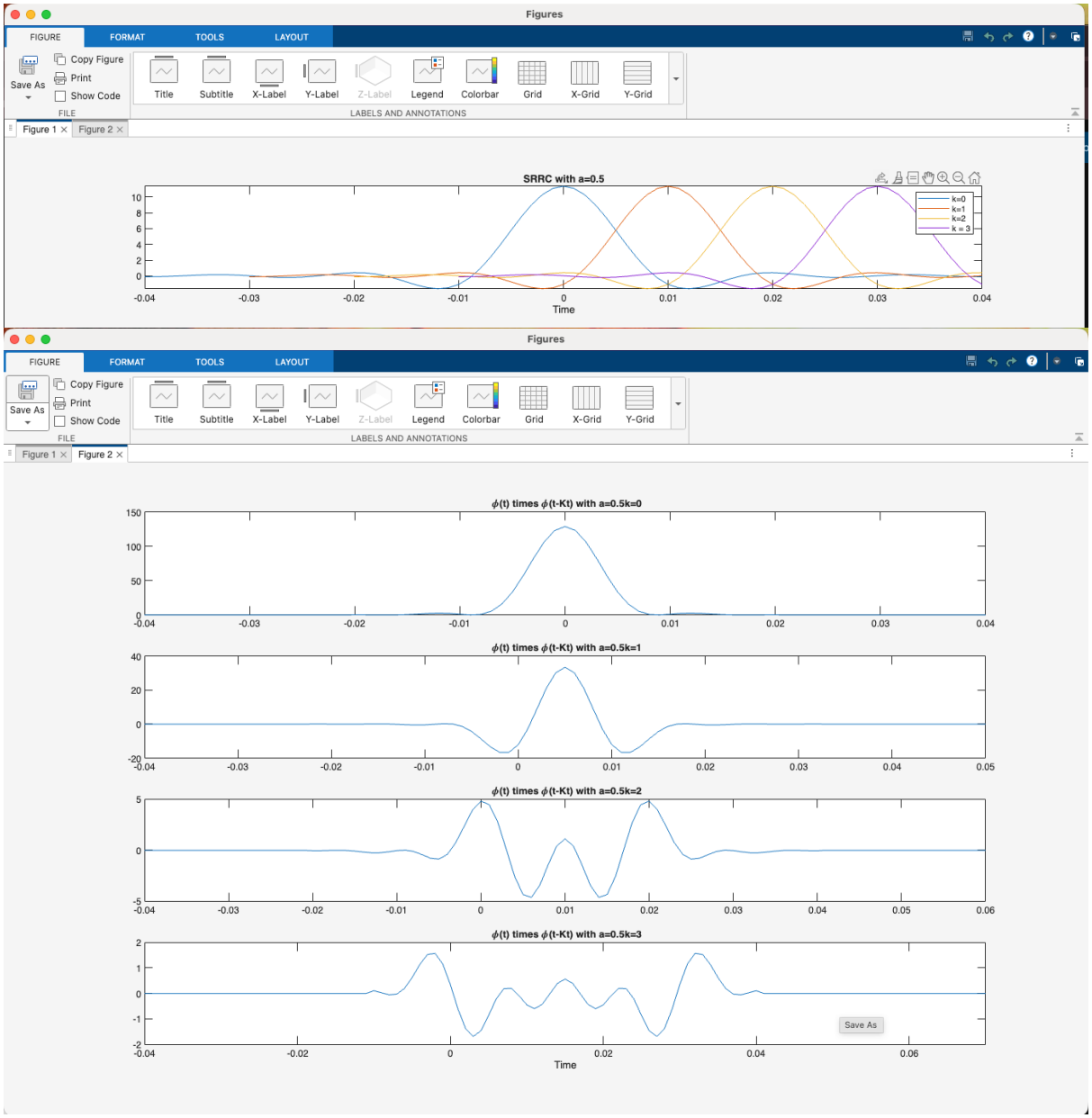
1. plot φ(t) and φ(t − kT) on the same plot,

*2. plot the product φ(t)*φ(t − kT),*

*3. numerically approximate the integral of the product φ(t)*φ(t − kT) using the method described in the notes.*

(10) Include in your report the results of steps 1 and 2 for a = 0.5 and k = 0, 1, 2, 3.

The results are plotted below:



(10) Include in your report the values of the integrals from step 3 for a = 0, 0.5, 1 and k = 0, 1, 2, 3, and try to explain them.

For α=0:

$\int \varphi(t) * \varphi(t - 0T)dt$= 0.974749

$\int \varphi(t) * \varphi(t - 1T)dt$= 0.029027

$\int \varphi(t) * \varphi(t - 2T)dt$= -0.034885

$\int \varphi(t) * \varphi(t - 3T)dt$= 0.046111

For α=0,5:

$\int \varphi(t) * \varphi(t - 0T)dt$= 0.999876

$\int \varphi(t) * \varphi(t - 1T)dt$= 0.000022

$\int \varphi(t) * \varphi(t - 2T)dt$= 0.000333

$\int \varphi(t) * \varphi(t - 3T)dt$= -0.000341

For α=1:

$\int \varphi(t) * \varphi(t - 0T)dt$= 0.999969

$\int \varphi(t) * \varphi(t - 1T)dt$= -0.000047

$\int \varphi(t) * \varphi(t - 2T)dt$= -0.000082

$\int \varphi(t) * \varphi(t - 3T)dt$= -0.000203

Conclusion:

The numerical integration of φ(t)*φ(t-kT) confirms that truncated SRRC pulses are approximately orthonormal. For k=0, the integral represents the pulse energy (which is close to 1), while for k =/ 0 the values approach 0 as expected. We notice that the approximation becomes much more accurate as α increases toward 1. This happens because a larger α causes the pulse tails to decay faster in time. Ultimately, while the ideal SRRC pulse is perfectly orthonormal, the practical version relies on a high α to minimize the overlap error and keep the shifts nearly orthogonal."

The MATLAB code used for this exercise is shown below:

B.m

/Users/kyrkos/Documents/MATLAB/MATLAB_LAB1/B.m

```
clear all;
close all;

%Question B1

T = 0.01; %Period
A = 4;
a = 0.5; % Roll - off factor
over = 10;
kStep = 1;
k = [0 ,1 ,2, 3];
t_delay = k .* T ;
Ts = T / over ; % sampling period
phi = {};

%Set vars for each roll-off factor and plot
for i =1: length(a)
    [phi{i}, t] = srrc_pulse(T,over,A,a);
    subplot(length(k),1,i);
    for j=1:length(k)
        plot(t+t_delay(j),phi{i});
        hold on;
    end
    title(['SRRC with a=' num2str(a)]);
    legend ('k=0','k=1','k=2','k = 3');
    xlabel('Time');
    xlim([-A*T A*T]);
end

hold off;

%φ(t)*φ(t-kT)

t_delay2=k*T;

for i=1:length(a)
    figure
    for j=1:length(k)
        subplot(length(k),1,j);
        time = -A*T:Ts:A*T+k(j)*T;
        phi_t=[phi{i} zeros(1,k(j)*T/Ts)];   %Zero pdding
        phi_KT = [zeros(1,k(j)*T/Ts) phi{i}]; %zero padding
        result = phi_t.*phi_KT; %Multiply
        plot(time,result);
        xlim([-A*T A*T+k(j)*T]);
        title([' \phi(t) times \phi(t-Kt) with a=' num2str(a(i)), 'k=' num2str(k(j))]);
    end
    xlabel('Time');
end

%ypologismos oloklиromatwn

k = [0 ,1 ,2, 3];
for i=1:length(a)
    fprintf('Integral of \phi(t)*\phi(t-kT) with a=%.1f and\n', a(i))
     for j=1:length(k)
        phi_t=[phi{i} zeros(1,k(j)*T/Ts)];
        phi_KT = [zeros(1,k(j)*T/Ts) phi{i}];
        result = phi_t.*phi_KT;
        v(i)=sum(result)*Ts;
        fprintf('k=%d: %.6f\n', k(j), v(i))
     end
end

```

C.1

Generate N bits bi, for i = 0, . . . ,N − 1 (e.g., N = 50, 100), using the command b = (sign(randn(N, 1)) + 1)/2;

The MATLAB code is shown below:

C.m ×

/Users/kyrkos/Documents/MATLAB/MATLAB_LAB1/C.m

```
1   clear all;
2   close all;
3
4   T = 0.01;
5   over = 10;
6   Ts = T/over;
7   a = 0.5;
8   A = 4;
9
10  % C.1
11
12  N = 50;
13  b = (sign(randn(N, 1)) + 1)/2;
14
```

C.2

The baseband 2-PAM system is implemented as follows.
1. (10) Write a function

$$X = bits\ to\ 2PAM(b);$$

which takes as input the bit sequence b and outputs the sequence of 2-PAM symbols X using the mapping:

$$0 -> +1,$$
$$1 -> -1.$$

The following function was created in MATLAB using the parameters set above. More in detail, if the input is 0 it converts it to +1 and if the input is 1 it converts it to -1.

C.m × bits_to_2PAM.m ×

/Users/kyrkos/Documents/MATLAB/MATLAB_LAB1/bits_to_2PAM.m

```
1  function X = bits_to_2PAM(b)
2
3      X = 1 - 2*b;
4
5  end
```

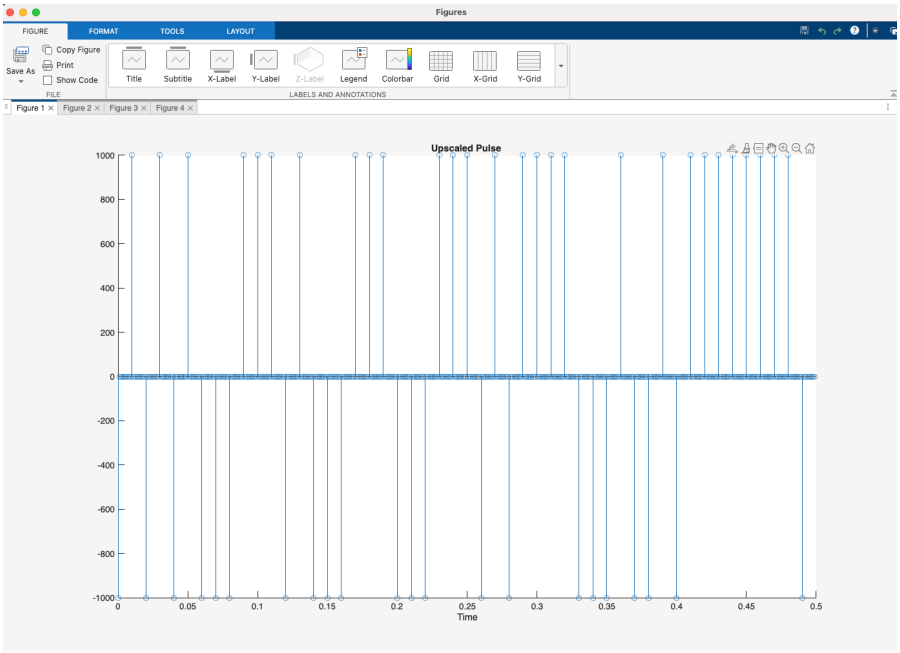
(*10) Simulate the signal*

$$X_\delta(t) = \sum_{k=0}^{N-1} X_k\, \delta(t - kT),$$

*using the command: X delta = 1/T s * upsample(X, over);*

Define the appropriate time axis and plot X(t).

The signal is simulated(plotted) below:



The MATLAB code is also shown below:

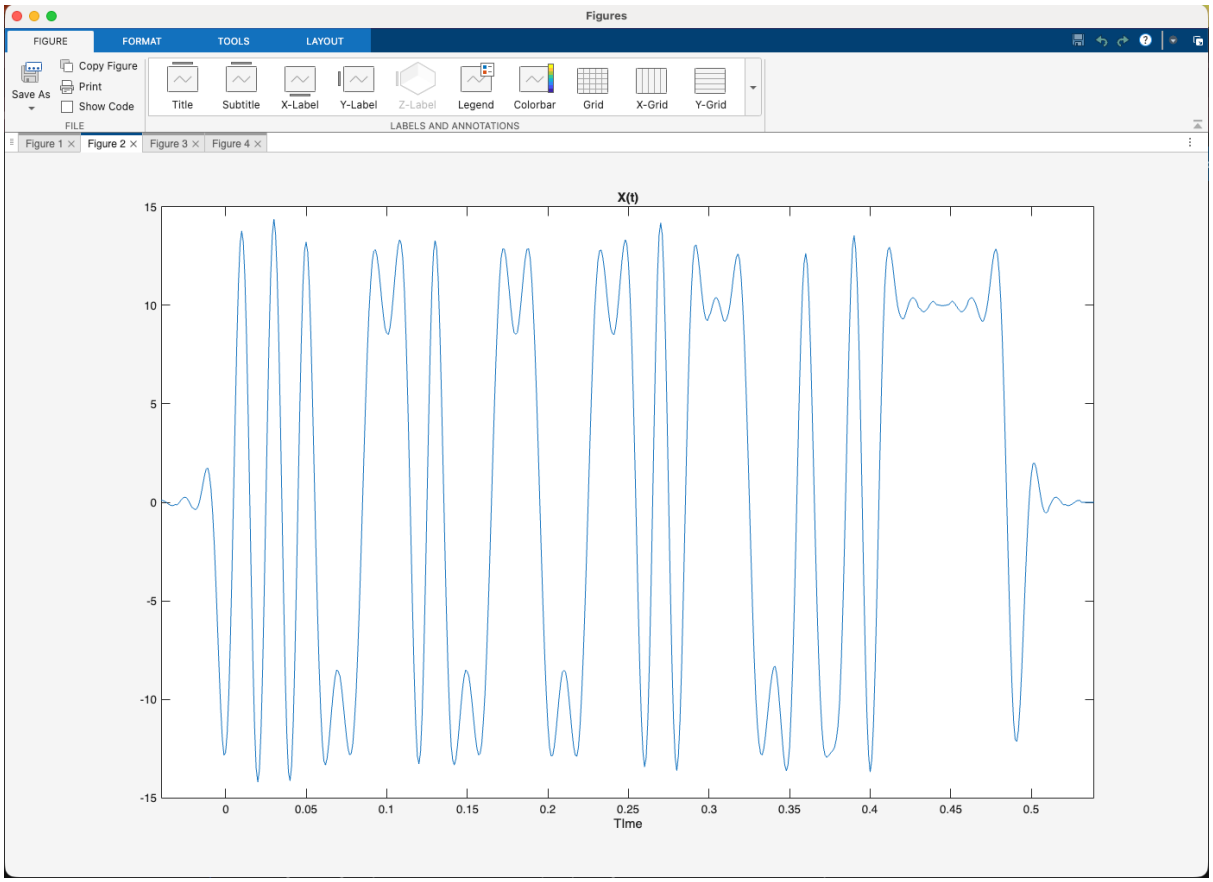
```
% C.2.2

x_delta = (1/Ts)*upsample(x, over);
t_pulse = 0:Ts:T*N-Ts;

figure
hold on;
stem(t_pulse, x_delta) ;
title("Upscaled Pulse ")
xlim([0 T*N]) ;
xlabel("Time") ;
```

(10) Create a truncated SRRC pulse φ(t) (using the parameters above) and simulate the convolution X(t) = Xδ(t) ⊛ φ(t). Construct the appropriate time axis and plot X(t).

Plot:



MATLAB Code:

```
% C.3

[phi, t] = srrc_pulse(T, over, A, a);
X_conv = conv(x_delta, phi)*Ts;
X_time = t_pulse(1)+t(1):Ts:t_pulse(end)+t(end);

figure
plot(X_time, X_conv)
xlabel("TIme")
xlim([X_time(1) X_time(end)]) ;
title("X(t)");
```

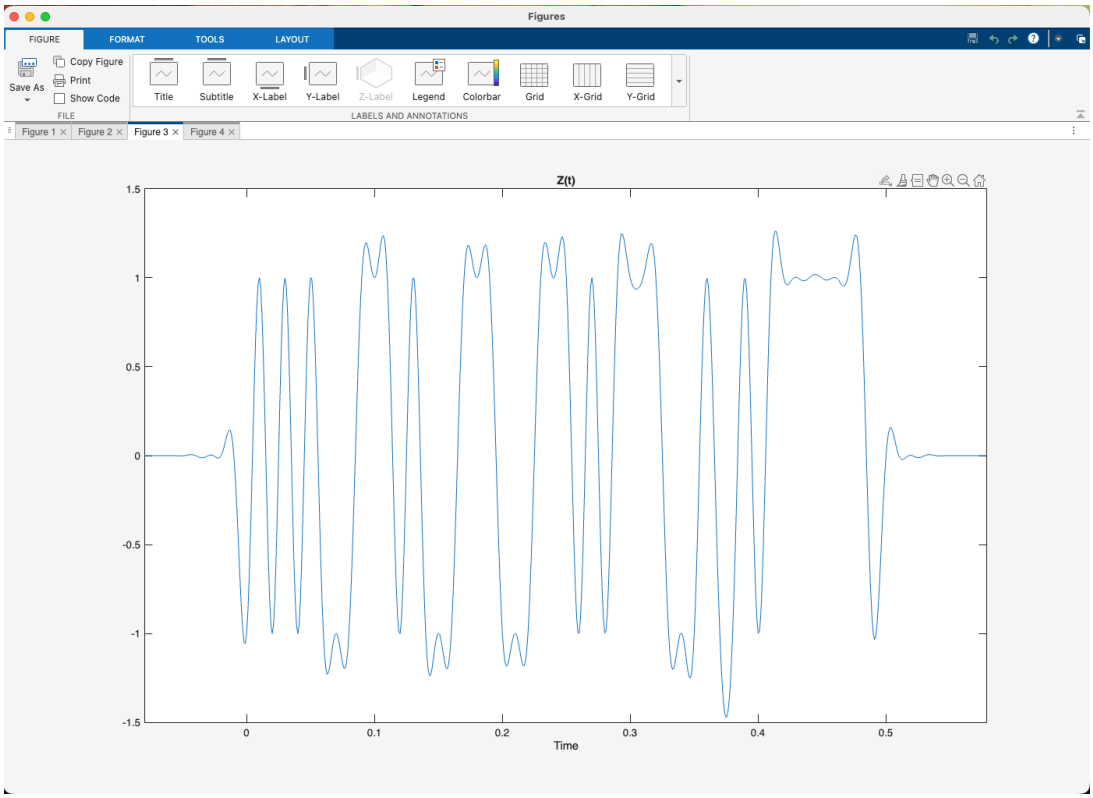
Assuming an ideal channel, the receiver input is X(t).

(10) Simulate the convolution Z(t) = X(t) ⊛ φ(−t).

Plot Z(t) over the corresponding time axis and find the values it takes at time instants kT, for k = 0, . . . ,N − 1. Relate these values to Xk, for k = 0, . . . ,N − 1.

Can you explain the phenomenon?

Plot:



MATLAB code:

```
% C.4.1

phi_flip = flip(phi);
Z_conv = conv(X_conv, phi_flip)*Ts;
Z_time = X_time(1)+t(1):Ts:X_time(end)+t(end);

figure
plot(Z_time, Z_conv)
xlabel("Time")
xlim ([Z_time(1) Z_time(end)]) ;
title("Z(t)");
```

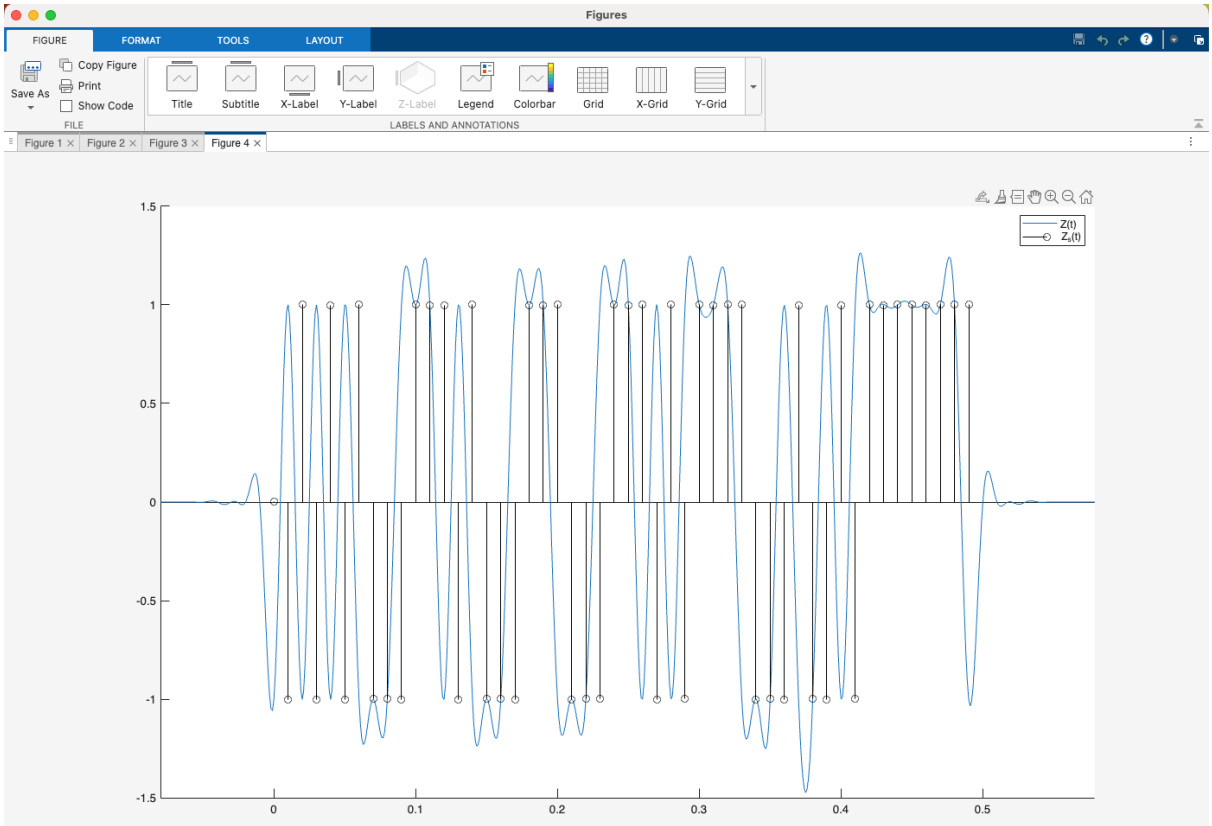
After calculating the convolution Z(t), we can see from the plots that the values at each sampling time kT are almost exactly the same as the original symbols X_k we sent. For example, when we transmitted a +1, the signal Z(t) at that specific moment is also approximately +1.This happens because the pulses are designed in a way that when we sample one symbol, all the other pulses before and after it are at zero. This means they don't interfere with each other, so we can recover the original data clearly.

(10) A graphical way to compare Z(kT) with Xk, for k = 0, . . . ,N − 1, is hold on the plot of Z(t) and execute

$$\text{stem}([0 : N - 1] \quad T, X);$$

where X is the vector of symbols Xk, k = 0, . . . ,N − 1. What do you observe?
Can you explain the phenomenon?

Plot:



MATLAB code:

```
% C.4.2

index_first = find(abs(Z_time) < T, 1);
index_last = find(abs(Z_time - t_pulse(end)) < T, 1);

Z_sampled = Z_conv(index_first : index_last);
Z_sampled = downsample(Z_sampled, over);

figure
hold on;
plot(Z_time, Z_conv);
stem(0:T:(N-1)*T, Z_sampled, "k");
legend ("Z(t)", "Z_s(t)");
xlim([Z_time(1) Z_time(end)]);
```

When we overlay the original symbols using the stem command on the plot of Z(t), we see that the stems land exactly on the peaks and values of the continuous wavy line. Specifically, at every time kT, the value of Z(t) is almost identical to the X_k symbol we sent (+1 or -1). When the transmitter pulse and the receiver filter are combined, they create a shape that hits zero at all other sampling points except for the current one. So, even though all the pulses are added together, at the exact moment t=kT, only the k-th symbol contributes to the value, and all others are zero. This is why we can recover the X_k values perfectly without any interference from the neighboring symbols.

Conclusion:

Through this exercise, we explored how the design of a communication pulse affects both bandwidth and signal recovery. We learned that while a low roll-off factor (a=0) makes the main frequency lobe narrower, it also causes the signal to decay very slowly in time. By simulating the full 2-PAM system, we saw that the SRRC pulse is a great practical choice because it balances these trade-offs. The most important thing we observed was the approximate orthonormality of the shifted pulses. By using a matched filter at the receiver, we were able to sample the signal at exactly t=kT and recover our original symbols without interference. Ultimately, we realized that in real systems, 'optimal' bandwidth depends on our specific requirements for pulse truncation.